

电机控制新平台软件简介

版本：v1.3.0



@ 2024, 版权归凌鸥创芯所有
机密文件，未经许可不可扩散

目录

1. 文档说明.....	5
2. 程序文件结构	6
2.1 1_LKS_FwLib 文件	7
2.2 2_HardwareDriverLayer 硬件驱动层文件	8
2.3 3_CommonServiceLayer 共用层文件	9
2.4 4_MotorDriveLayer 电机驱动层文件	10
2.5 5_MotorAppLayer 电机应用层文件.....	11
2.6 6_UserAppLayer 用户应用层文件	13
2.7 Include 配置头文件.....	14
3. 控制对象框图	15
4. 共用层控制对象.....	17
5. UA 层控制对象	20
6. MA 控制对象.....	23
7. MD 层控制对象.....	27
8. HD 层配置.....	29
9. 数据格式.....	30
9.1 标幺定义.....	30
9.1.1 电流标幺	31
9.1.2 电压标幺	32
9.1.3 速度标幺	32
9.1.4 角度标幺	32
9.2 标幺函数.....	34
9.3 数据表示.....	35
10. 功能简介.....	37
10.1 VF 启动	37
10.2 自动过调制.....	37
10.3 SVPWM.....	38
10.4 自动弱磁	38
10.5 参数自动计算	38
10.6 无速度观测方式.....	38
10.7 电流环解耦补偿.....	38

10.8 快速闭环启动	39
10.9 锁相环参数调整.....	39
10.10 母线电压保护功能.....	39
11.库函数	40
11.0 状态检查函数.....	40
11.1 VF 函数	40
11.2 电流设置及弱磁控制函数.....	41
11.3 低压保护异常处理函数.....	42
11.4 无速度观测器函数.....	43
11.5 顺逆风检测函数	45
11.7 IPD 函数	45
11.8 SVPWM 函数	46
11.9 PI 参数自计算函数.....	47
11.10 反电势电路检测函数.....	47
11.11 有感处理函数函数.....	48
11.12 参数查看函数.....	48
12. 报警处理内容	51
13. 调试步骤.....	52

电机控制新平台软件简介

1. 文档说明

本文档仅是对电机控制新平台软件简明概要手册, 具体新算法平台的详细开发与调试介绍请参考算法新平台使用手册, 实际软件版本若有迭代更新, 以软件为主。

当前文档内容及介绍以 LKS32MC08x 为例, 其他类型 MCU 可以按照相同的软件架构进行参考和移植。

库函数部分按照最新软件更新, 可能存在旧版本不支持某些函数的情况, 以软件为准。

文件说明	新平台软件
版本	V1.0
日期	2023.05.15
适用 MCU	LKS32MC08x/LKS32MC03x/LKS32MC45x
编制	Lith
文件说明	新平台软件
版本	V1.1(update)
日期	2023.11.09
适用 MCU	LKS32MC08x/LKS32MC03x/LKS32MC45x
编制	Lith
文件说明	新平台软件
版本	V1.2.0(update)
日期	2023.11.29
适用 MCU	LKS32MC08x/LKS32MC03x/LKS32MC45x
编制	Lith
文件说明	新平台软件
版本	V1.3.0(update)
日期	2024.12.16
适用 MCU	LKS32MC08x/LKS32MC03x/LKS32MC45x
编制	Lith

2. 程序文件结构

当前程序文件夹结构如下：

1_LKS_FwLib	MCU底层驱动库
2_HardwareDriverLayer	硬件层，包含硬件初始化程序以及抽象驱动函数
3_CommonServiceLayer	共用层，包含各层数据交换接口，参数配置等
4_MotorDriveLayer	电机驱动层，电机驱动FOC的实现
5_MotorAppLayer	电机应用层，电机控制状态机及三环实现 报警检测及处理
6_UserAppLayer	用户应用层，包含用户状态机，用户控制命令等
Include	配置头文件等

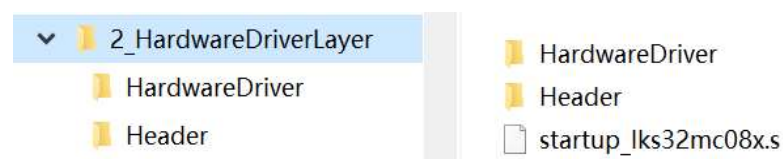
2.1 1_LKS_FwLib 文件

包含选用的 MCU 驱动库，如 lks32mc08x_periph_driver

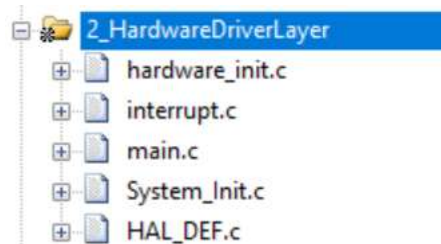
根据选用的 MCU，包含所需的硬件驱动库函数，并修改包含 MCU 头文件相应文件。

MCU 底层库文件或驱动升级时需要检查硬件驱动层（HD）软件的适配性及函数接口一致性。

2.2 2_HardwareDriverLayer 硬件驱动层文件



包含以下文件



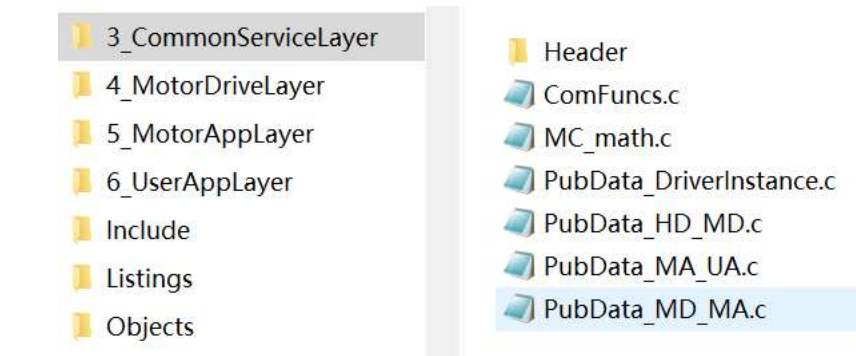
其中，HAL_DEF.c 中定义了硬件驱动抽象函数，用于电机驱动层（MD 层）实现接口调用。

hardware_init.c 函数用于硬件初始化，部分硬件驱动初始化函数可以根据存储在 FLASH 参数空间的参数进行相应的配置，如 PWM 周期，OPA 放大倍数的选择，DAC 比较电压的选择等。因此电机驱动层会应用到共用层 3_CommonServiceLayer 的参数配置信息。

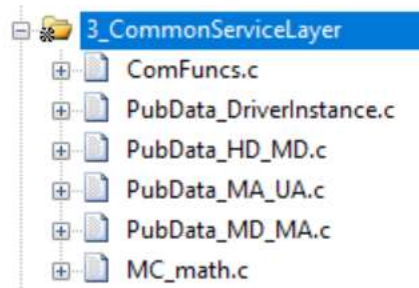
main.c:	系统主函数。
interrupt.c:	用于中断子程序函数的处理。
startup_lks32mc08x.s:	汇编函数包含 mcu 的中断向量定义等 根据 MCU 选用相应的函数。

硬件驱动层(HD 层)跟硬件相关，原理图不同时，需要修改硬件驱动层初始化函数和相应的硬件驱动抽象函数，进行相应的外设驱动初始化和驱动函数的修改。

2.3 3_CommonServiceLayer 共用层文件



包含以下文件：



共用层用于定义各层公用的数据结构和相互的交换信息。

- ComFuncs.c : 定义标么函数等
- MC_math.c : 共用的数学计算实现

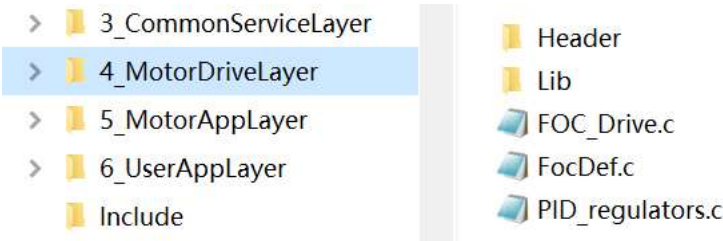
PubData_DriverInstance.c :
用于定义共用控制对象、各层交换数据、配置参数信息实例等

- PubData_HD_MD.h : 定义 HD 与 MD 层信息交换数据结构
- PubData_MA_UA.h : 定义 MA 与 UA 层信息交换数据结构
- PubData_MD_MA.h : 定义 MA 与 MD 层信息交换数据结构
- PubData_DriverInstance.h : 定义共用控制对象数据结构，

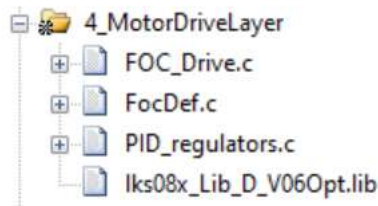
该层为各层共用信息和数据结构接口，用于各层的函数调用以及命令定义。为保证程序的一致性和兼容性等，实际应用中不可随意修改 CommonServiceLayer 下的定义。

由于 mcu 的不同，实现数学函数的计算方式不同，部分数学函数要根据 MCU 有针对的实现。

2.4 4_MotorDriveLayer 电机驱动层文件



包含以下文件：



该层用于电机 FOC 实现。

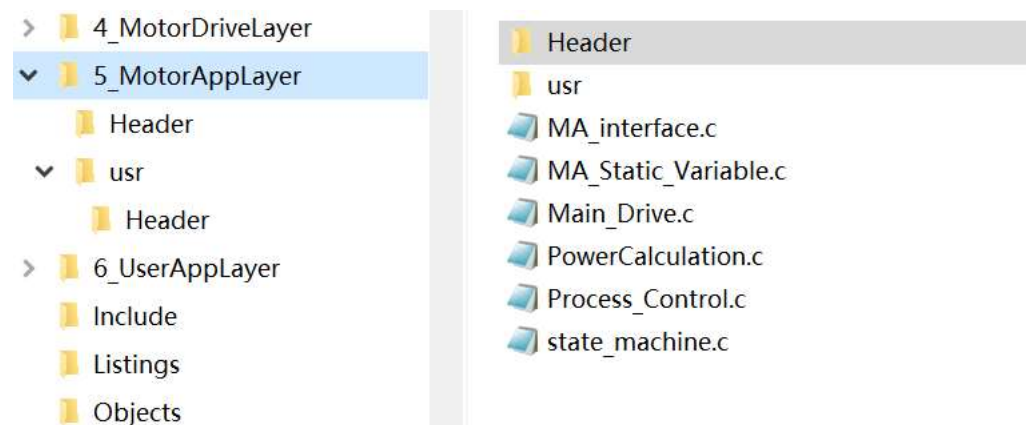
其中：

- FocDef.c : FOC 相关函数的实现
- FOC_Drive.c : FOC 程序状态机及相关数据交换函数、操作命令的实现
- PID_regulators.c : 电流环 PI 的实现

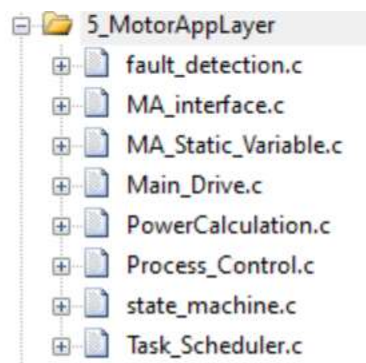
- xxxxxxx.lib : 电机控制库函数，不同应用包含不同的头文件
不同版本时采用不同的库文件，以实际版本文件为准

- mc_declaration.h : MD 层数据结构及库函数声明

2.5 5_MotorAppLayer 电机应用层文件



包含一下文件：



其中：

MA_interface.c	: MA 层对外接口函数，用于外部调用等
MA_Static_Variable.c	: MA 内部变量实例定义函数
state_machine.c	: MA 状态机控制函数
Main_Drive.c	: 各状态机下 状态处理函数
PowerCalculation.c	: 功率计算及处理函数

usr 子文件夹下包含下面的文件，便于用户进行修改

fault_detection.c	: 报警检测及处理函数， 根据实际应用修改
Task_Scheduler.c	: 调度任务函数

MA 下的状态机已经包含了目前的预充电、顺逆风、IPD、对相、启动、停止以及报警处理的逻辑处理，状态机的切换可以通过宏编译、或者参数配置实现，因此如无必要可以不修改状态机，通过参数配置实现状态切换。

如： 如果不需要 align 对相功能时，可以有以下三种方式实现：

方法一： 配置宏 ALLIGN_STATUS_MO 为 FALSE，程序执行时自动跳过对相

电机控制新平台软件简介

方法二： 宏 `ALIGN_STATUS_M0` 为 `TRUE` 时配置对象时间为 0

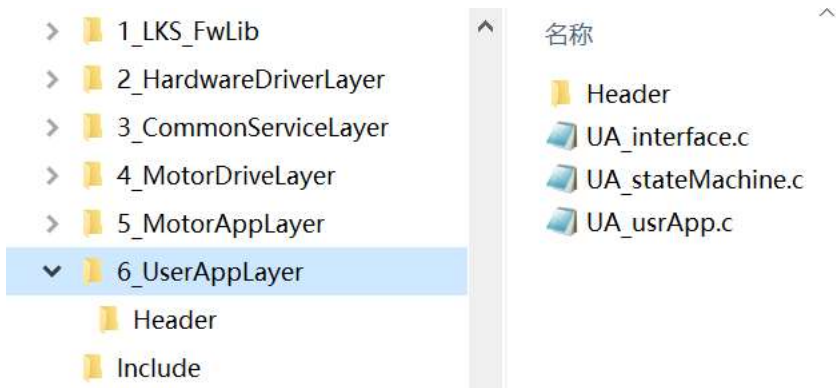
程序执行到 `align` 状态时时间到达，自动跳过对相到下一步状态机

方法三： 修改代码，修改状态机跳转逻辑中的部分代码。

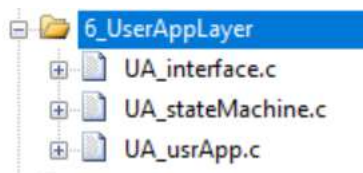
实际应用时，建议采用方法一，其次为二，最后是采用方法三。

MA 层详细的功能描述参阅相关文档。

2.6 6_UserAppLayer 用户应用层文件



包含文件：



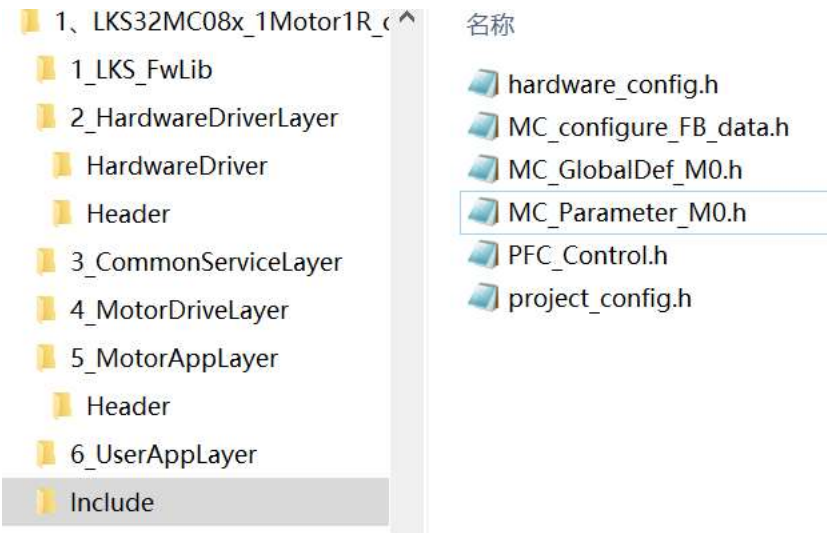
- UA_interface.c : UA 层初始化、状态机任务调度对外接口函数
- UA_stateMachine.c : UA 层状态机及控制命令管理
- UA_usrApp.c : UA 用户程序文件

该层用于用户根据特定需要进行电机控制，该层可以选择电流模式、速度控制模式或者功率模式，**用户可以根据需要进行相应的代码处理，添加特定的功能或相应的函数。**

在单驱动单元时，代码根据控制模式用宏定义进行了预编译处理；在双的驱动单元时，所选用的 MCU 具有足够大的 FLASH 空间，代码全部编入，运行过程中可以在应用层进行控制模式的选择，默认在 stop 状态下进行控制模式更改，如需在运行中进行更改用户可以添加用户代码实现。

所有的电机信息，如转速、电压、电流、功率、报警信息均可查看，并且相应的用户层状态机可以进行相应的状态维护。

2.7 Include 配置头文件



包含工程配置文件、参数配置文件等信息

其中：

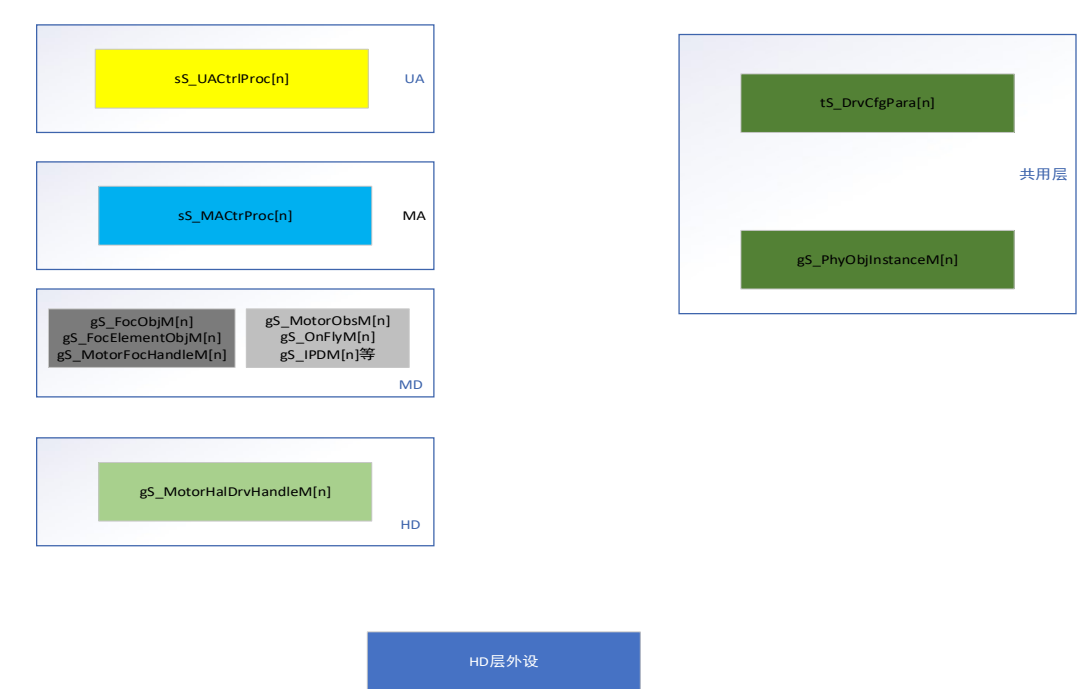
- project_config.h ： 工程配置文件，
包含参数组个数，PWM 模块个数等关键信息
- hardware_config.h ： 硬件配置文件

MC_GlobalDef_M0.h ： 电机 0 全局配置信息，如开关频率、采样方式、采用的 PWM、是否启用对相等功能的配置、死区时间、电机停止方式、是否允许报警自动清除等……………

MC_Parameter_M0.h ： 电机控制相关参数宏定义

MC_configure_FB_data.h ： 配置参数数据结构定义
用于将 MC_GlobalDef_M0.h 和 MC_Parameter_M0.h 的配置参数生成数据结构，然后编译器根据 project_config.h 指定的地址空间 link 到固定的 FLASH 扇区中，实现与 PC 软件的共地址数据读取。

3. 控制对象框图



共用层、UA 层、MA、MD 层以及 HD 层的抽象驱动函数对象通过数据结构下标索引，即数据结构的下标为当前对象的 ID，同一下标下的对象为同一电机控制的数据实例。

`sS_UACtrlProc[n]`、`sS_MACtrlProc[n]`、`gS_PhyObjInstanceM[n]`，以及 MD 层控制对象的实例个数由 `MAX_PWM_MODULE_USED` 宏决定。

MD 驱动函数接口：

```
#if (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_2SHUNT)
#define Motor_FocMethod_M0 \
{ /* FOC Methods */ \
  SVPWM_2Shunt_Enl, \
  /* SVPWM调制 */ \
}
#elif (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_1SHUNT)
#define Motor_FocMethod_M0 \
{ /* FOC Methods */ \
  SVPWM_1Shunt, \
  /* SVPWM调制 */ \
}
#else
#define Motor_FocMethod_M0 \
{ /* FOC Methods */ \
  SVPWM_3Shunt, \
  /* SVPWM调制 */ \
}
#endif
```

MD 层驱动函数定义

```
#if (MAX_PWM_MODULE_USED == 1)
const STR_MotorFocHandle
#elif (MAX_PWM_MODULE_USED == 2)
const STR_MotorFocHandle
#endif

gS_MotorFocHandleM[MAX_PWM_MODULE_USED] = {
  Motor_FocMethod_M0
};

gS_MotorFocHandleM[MAX_PWM_MODULE_USED] = {
  Motor_FocMethod_M0,
  Motor_FocMethod_M1
};
```

硬件抽象驱动函数如下：

```
#if (MAX_PWM_MODULE_USED == 1)
const STR_HALDrvHandle  gS_MotorHalDrvHandleM[MAX_PWM_MODULE_USED] = {Motor_HALDrv_EPWM0
                                                                    };
#elif (MAX_PWM_MODULE_USED == 2)
const STR_HALDrvHandle  gS_MotorHalDrvHandleM[MAX_PWM_MODULE_USED] = {Motor_HALDrv_EPWM0,
                                                                    Motor_HALDrv_EPWM1
                                                                    };
#endif
```

其中驱动函数指针数组如下

```
#define Motor_HALDrv_EPWM0 \
{
    /* DRV handle */
    AcquireCurSamplingResultsM0, /* 读取电流AD采样 */ \
    AcquireVdcSamplingResultsM0, /* 读取母线AD采样 */ \
    AcquireNTCSamplingResultsM0, /* 读取NTC AD采样 */ \
    AcquireBmefSamplingResultsM0, /* 读取BMEF AD采样 */ \
    ChangeADChanelCFGMO, /* 切换AD采样通道 */ \
    MCPWM0_RegUpdate, /* PWM加载 */ \
    EPWM0_OutPut, /* PWM输出使能 */ \
    EPWM0_Charge, /* PWM预充电 */ \
    InitEPWM0ChargeEnd, /* PWM预充电初始化 */ \
    GetEPWM0_breakInStatus, /* 读取 BreakIn 状态 */ \
    ClrEPWM0_breakInStatus /* 清除 BreakIn 状态 */ \
}
```

共用层对象 gS_PhyObjInstanceM[]中的成员 **m_bPWM_Id** 为当前控制对象组采用的实际物理端口对应的 PWM 组及相应的 AD 端口配置组。

查找控制对象有以下方式：

- 1) UA、MA 层通过下标可以直接查找到当前对象的指针，下标即对象的 ID

调用查找对象函数

同时可以根据索引指针查找到 gS_PhyObjInstanceM 对象，进而查找到当前应用的 PWM_ID，得到当前应用的是 EPWM0 或 EPWM1。

对单电机控制的 MCU，默认为 EPWM0

- 2) 查找 PWM 模块对应的控制对象指针

方式一：

MA 层调用 getMAIDFromPWMID () 返回 MA 对象的 ID，

因 MA 与 UA、MD 对相、共用层对象的 ID 都是其数组下标，即 sS_UACtrlProc[n] 与 sS_MACtrlProc[n]、gS_PhyObjInstanceM[n] 对应，可调用相应的 get 对象函数获得对象指针。

方式二：

直接遍历相应的对象数组，通过比较对象指针索引的 gS_PhyObjInstanceM 数据结构的 m_bPWM_Id 成员后获得对象的 ID。

4. 共用层控制对象

gS_PhyObjInstanceM[MAX_PWM_MODULE_USED]

其中：

```
// 支持的 PWM 通道数据，即可支持驱动单元数目，由应用和 MCU 所支持的 PWM 模块单元决定
#define MAX_PWM_MODULE_USED (PLANTFORM_MAX_PWM_MODULE_USED)
----- 由 project_config.h 中的宏定义指定
```

控制参数组

tS_DrvCfgPara[MAX_CFG_PARA_ID]

```
// 常量 FB 配置参数结构体组数，个数由应用决定，可与硬件 PWM 模块个数独立，不相关
#define MAX_CFG_PARA_ID (PLANTFORM_MAX_CFG_PARA_ID)
----- 由 project_config.h 中的宏定义指定
```

调用函数

u8 getPWMIDUsed(u8 objID)	获取当前应用的 PWM 号，EMPW0 或 EPWM1
void BondingMotorDrvObject(void)	初始化共用层数据结构
bool reBondingMotorDriveParaDat(u8 objID, u8 objCfgID)	控制参数重绑定
PSTR_ControlObject getCtrlObjectPtr(u8 objID)	根据对象 ID 获取共用层对象指针

gS_PhyObjInstanceM 数据结构如下：

```
typedef struct {
    u8 m_bPWM_Id; //实际的PWM物理端口号，0 ----- EPWM0
    /* MA2MD FOC参数 */
    PSTR_MotorFocParDef m_pMotorFocPar;
    /* 添加参数配置结构体指针，配置参数与控制对象绑定，const数据类型 */
    PSTR_DrvCfgPara m_pDrvCfgPara;
    /* Trans Coef for application, Used by all layer */
    STR_TransCoef mTransCoef;
    /* Definition about global data structure interface between MA and MD */
    STR_GlobalDatPackage mGlobalDatPackage;
}STR_ControlObject,*PSTR_ControlObject;
```

各层间数据结构，如硬件驱动层和电机驱动层

电机控制新平台软件简介

```
typedef struct {
    bool    bBreakInStatus;    // 是否存在过流

    int16_t nSampCurDat0;      // 电流采样值0
    int16_t nSampCurDat1;      // 电流采样值1

    int16_t nPhaseUOffset;
    int16_t nPhaseVOffset;
    int16_t nPhaseWOffset;
    int16_t nBusShuntOffset;

    int16_t nDcVolAdcDat;      // 母线采样值

    int16_t nSampDatU;         // iu
    int16_t nSampDatV;         // iv
    int16_t nSampDatW;         // iw

    /*反电势检测*/            // 条件编译
    #if 0
    int16_t nNTCVolAdcDat;      // NTC采样值
    int16_t nBmefUVolAdcDat;    // U反电势
    int16_t nBmefVVolAdcDat;    // V反电势
    int16_t nBmefWVolAdcDat;    // W反电势
    #endif
}STR_PubDatHDMD;

typedef struct {
    uint16_t nuTimePhAN;
    uint16_t nuTimePhA;
    uint16_t nuTimePhBN;
    uint16_t nuTimePhB;
    uint16_t nuTimePhC;
    uint16_t nuTimePhCN;
}STR_McpwmCmpr, *PSTR_McpwmCmpr; /*电机驱动层返回值*/

typedef struct {
    uint16_t          nuPoint0;
    uint16_t          nuPoint1;
    ENUM_ADChannelCFG eChannel;
}STR_McpwmTmr, *PSTR_McpwmTmr; /*电机驱动层返回值*/

typedef struct {
    STR_McpwmTmr      mMcTmr;
    STR_McpwmCmpr      mMcPwm;
}STR_PubDatMDHD;
```

MA 与 MD 层:

```
typedef struct {
    int16_t          gWrFlag;            // 互斥变量
    ENUM_MotorStatus eMotorStatus;       // MD层状态指令
    STR_VectorDQ      mStatCurDQCmd;     // DQ电流指令
    int32_t          wMotorSpeedRef;     // 速度指令值

    ENUM_PWMStatus    ePwmCmd;           // PWM操作命令 ON---MOE使能
    int16_t          nMoeActionBlankTime; // 消隐时间5ms..
    STR_MotorObsParOnLine mMotorParRI;
    uint8_t          bIpdStart;          // IPD使能
    int16_t          nMotorBrakePwmDuty;  // 制动宽度 q12
    ENUM_FineCmd      nuFineCmd;         // 0:none
                                           // E_MOTOR_RESET: 0x01: init Acr
                                           // E_MOTOR_TRACKING E_MOTOR_ALIG
                                           // 0xF0: 0xF1: ...
                                           // E_MOTOR_BRAKE, E_MOTOR_IPD
                                           // 0x0100:Duty brake 0x0101:s

}STR_PubDatMAMD, *PSTR_PubDatMAMD; // MA命令数据结构(发送给 MD)
```

电机控制新平台软件简介

```
typedef struct {  
    int16_t  
    ENUM_PWMStatus  
    bool  
  
    int32_t  
    int16_t  
    STR_VectorDQ  
    STR_VectorDQ  
    STR_VectorUVW  
    int16_t  
    int16_t  
    int16_t  
    int16_t  
  
    ENUM_MotorStatus  
  
    ENUM_FineCmd  
} STR_PubDatMDMA;  
  
gWrFlag; //互斥标志  
ePwmStatus; //moe状态  
bBreakInFault; //BreakIn标志  
  
wFreq; //反馈频率值(复用为反电势)  
nVdc; //母线电压  
mStatCurDQ; //DQ电流值  
mStatVolDQ; //DQ电压值  
mStatCurrUVW; //定子电流UVW  
nMotorCur; //电机相电流有效值  
nMotorVol; //电机电压(复用为反电势检测)  
nPowerDec; //估算功率  
nTemp; //温度  
  
eMotorStatusAck; //MD层状态  
  
nuFineStatus; //0:none  
// MD状态数据结构(返回给 MA)
```

5. UA 层控制对象

```

/*****
//局部静态变量定义区
STR_UACtrlProc  sS_UACtrlProc[UA_OBJ_MAX_DEFINE];

#define UA_OBJ_MAX_DEFINE  (MAX_PWM_MODULE_USED)

```

UA 对象个数由硬件支持的 PWM 模块数目定义

外部可调用函数：

void InitUsrAppLayer(void)	UA 对象初始化
void UACtrlObjTask(void)	UA 控制对象任务处理函数，1ms 任务调用
pSTR_UACtrlProc get_UA_CtrPtr(u8 ObjID)	根据 ID 号查找 UA 对象指针
u8 get_Max_UAObj_Nbr(void)	获取 UA 对象数目

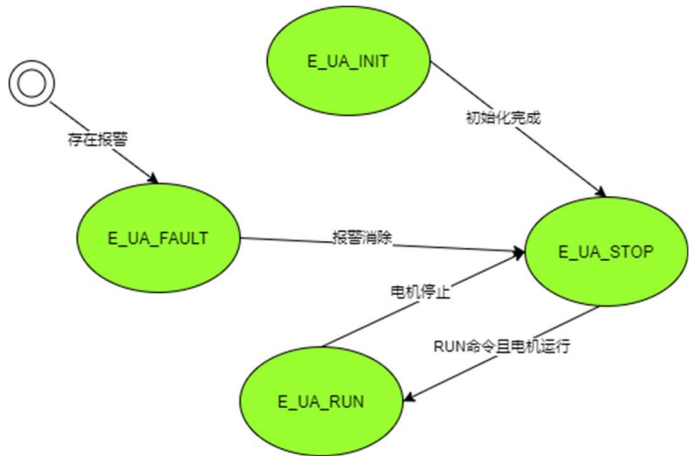
内部数据结构及状态机等：

应用程状态机

```

typedef enum
{
    E_UA_INIT           = 0,    /* UA状态机初始状态 */
    E_UA_STOP           = 1,    /* UA状态机停止状态 */
    E_UA_RUN            = 2,    /* UA状态机运行状态 */
    E_UA_FAULT          = 3,    /* UA状态机故障状态 */
    E_UA_ILLG           = 4,    /* UA状态机非法状态 */
} ENUM_UACtrlState;

```



UA 数据结构如下，用户可以根据需要自己添加相关信息，自行维护

电机控制新平台软件简介

```
typedef struct
{
    // 绑定的控制对象
    u8 mg_nUACtrlProcID; /*当前UA控制对象 ID*/
    PSTR_ControlObject mg_pCtrlObj; /*控制对象*/

    // 内部交换信息存储及刷新
    STR_PubDatUAMA mUaToMa;
    STR_PubDatMAUA mMatoUa;

    ENUM_UACtrlState m_eUASysState; /* 当前UA运行状态机 */
    STR_UA_CtrCMD m_StruCmnd;

    u8 m_bStopMode; /*电机停止模式*/
    u8 m_bPWMStopMode;
    u8 m_bLoopMode; /*运行模式*/

    s32 m_wSetpoint; /* 负值为反向 0.01Hz */
    s32 m_wTqSetpoint; /* 负值为反向 0.001A */
    s32 m_wPowerSetpoint; /* 0.01W */

    u8 m_bFaultAutoClr;
    u16 m_nuFaultClrCnt; /*故障清除计数*/
    u16 m_nuFaultClrPeriod; /*故障清除周期*/
}STR_UACtrlProc, *pSTR_UACtrlProc;
```

UA2MA 的交互信息字段包含有 wFreqCmd、wPowerCmd、wTorqCmd，UA 应用程序可以根据需要分别设置相应的命令值。

```
*****
函数名称: void updateUASetpoint(pSTR_UACtrlProc pObj)
功能描述: 单电机UA指令处理函数
输入参数: pSTR_UACtrlProc pObj
输出参数: 无
返回值: 无
其它说明: 用户可根据需要添加相应的处理函数或者处理方式
修改日期: 版本号 修改人 修改内容
-----
2022/8/19 V1.0 Tonghua Li 创建
2022/3/29 V1.1 Tonghua Li 更新
*****
void updateUASetpoint(pSTR_UACtrlProc pObj)
{
    PSTR_DrvCfgPara pDrvCfgPara = pObj->mg_pCtrlObj->m_pDrvCfgPara;

    #if (DRV0_CLOSE_LOOP == SPEED_LOOP) //速度环
    #if (DRV0_POWER_LIMIT_STATUS == TRUE) //速度环限功率
        pObj->m_wSetpoint = pDrvCfgPara->mS_FBSpdLoop.m_wPowerLmtSpdValue;
        pObj->m_wPowerSetpoint = pDrvCfgPara->mS_FBSpdLoop.m_wPowerLmtValue;
    #endif

    pObj->m_wTqSetpoint = pDrvCfgPara->mS_FBCurLoop.m_wIqSet;

    #elif (DRV0_CLOSE_LOOP == CURRENT_LOOP) //电流环
        pObj->m_wTqSetpoint = pDrvCfgPara->mS_FBCurLoop.m_wIqSet;

    #elif (DRV0_CLOSE_LOOP == POWER_LOOP) //功率环
        #if (DRV0_SPPED_LIMIT_STATUS == TRUE) //功率环限速
            pObj->m_wPowerSetpoint = pDrvCfgPara->mS_FBPwrLoop.m_wLmdSpdPowerSetting;
            pObj->m_wSetpoint = pDrvCfgPara->mS_FBPwrLoop.m_wSpdLmtValue;
        #else
            pObj->m_wPowerSetpoint = pDrvCfgPara->mS_FBPwrLoop.m_wPowerSetting;
        #endif

        pObj->m_wTqSetpoint = pDrvCfgPara->mS_FBCurLoop.m_wIqSet;
    #endif

    pObj->mUaToMa.wFreqCmd = pObj->m_wSetpoint; //转速指令设定
    pObj->mUaToMa.wPowerCmd = pObj->m_wPowerSetpoint; //功率指令设定
    pObj->mUaToMa.wTorqCmd = pObj->m_wTqSetpoint; //电流指令设定
}
```

UA 控制命令字段 STR_UA_CtrCMD 定义:

```
typedef struct
{
    u8 m_bRunFlag      :1;    /* UA 运行命令标志*/
    u8 m_bDirRevse     :1;    /* 方向取反命令标志*/
    u8 m_bFault        :1;    /* UA 故障标志*/
    u8 m_bFaultClr     :1;    /* UA 故障清除命令*/
    u8 m_bMoeEnable    :1;    /* MOE 使能标志*/
    u8 m_bDownLoadStart :1;    /* 开始下载*/

    u8 m_bResvd       :2;
}STR-UA_CtrCMD,*pSTR-UA_CtrCMD;
```

该字段用于 UA 内部控制命令处理，在发送给 MA 控制命令时，调用

UARunCmdHandle ()

UAStopCmdHandle ()

UAClrFaultCmdHandle ()

UAStoppingCmdHandle ()

UAFaultStopHandle ()

进行 UA2MA 命令的封装，按照 PubData_MA-UA.h 中定义的交互数据结构进行封装。

解析 MA2UA 的状态及信息时，调用 PubData_MA-UA.h 中定义的 inline 函数进行判断。

如：

```
static __inline int16_t getVdcDecMAUA(STR_PubDatMAUA* pMA2UA)
```

```
static __inline int16_t getCurDecMAUA(STR_PubDatMAUA* pMA2UA)
```

```
static __inline int16_t getSpeedDecMAUA(STR_PubDatMAUA* pMA2UA)
```

等

6. MA 控制对象

```
sS_MACtrProc[MA_OBJ_MAX_DEFINE];
```

```
/* MA 控制对象 后续更改为局部静态变量，外部不可直接调用*/
```

```
sS_MASubBlock[MA_OBJ_MAX_DEFINE];
```

```
/* MA 控制结构体 后续更改为局部静态变量，外部不可直接调用*/
```

调用函数：

```
void InitMotorAppLayer(void)
```

MA 对象初始化

```
u8 getMAIDFromPWMID(u8 PWMID)
```

根据 PWM_ID 查找 MA 的 ID 号

```
void MACtrlObjTask(void)
```

MA 控制对象任务处理函数，1ms 任务调用

```
PSTR_MACtrProc getMACtrPtr(u8 ObjID)
```

根据 ID 号查找 MA 对象指针

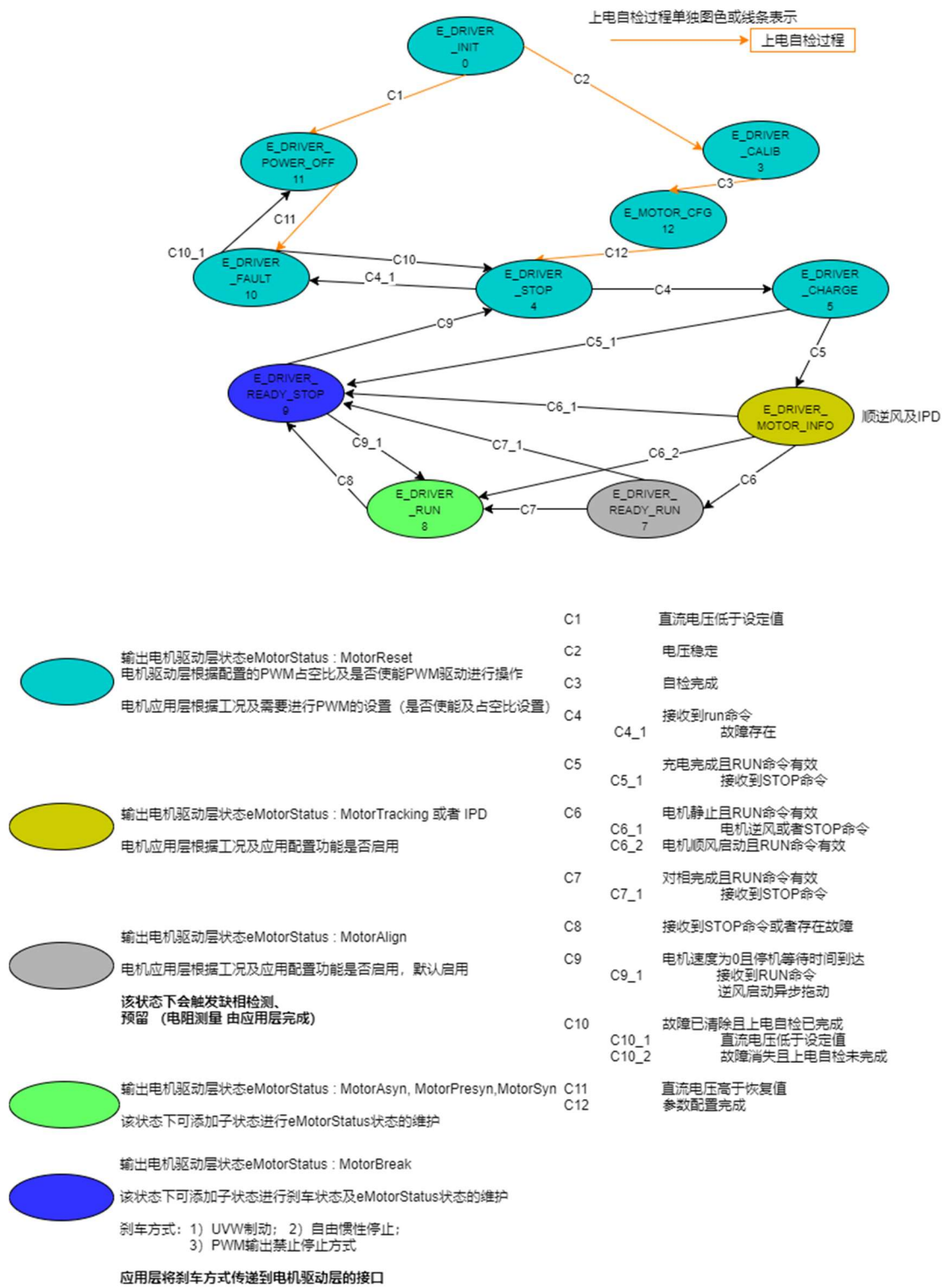
```
u8 getMaxMAObjNbr(void)
```

获取 MA 对象数目

调试的相关信息均可以通过 sS_MACtrProc

电机控制新平台软件简介

内部数据结构及状态机：



数据结构定义在 MA_Static_Variable.h 中，

子模块数据结构定义

电机控制新平台软件简介

```
typedef struct
{
    STR_Time_t          m_struTime;
    STR_CommData        m_struAppCommData;          /* 运行流程控制结 */
    STR_MechQuantity    m_struMotorSpeed;          /* 转速矢量结构体 */
    //stru_PIRegulator   m_SpeedPI;                /* 速度环 */

    #if ((CLOSE_LOOP_M0 == POWER_LOOP) || ((POWER_LIMIT_STATUS_M0 == TRUE
    STR_PowerDef        m_struPower;                /* 输入功率计算 */
    #endif

    STR_OnTheFlyDetectMA m_OnTheFlyDetect;

    #if(U_UNDVOLT_PROT_ENABLE_M0 == 1)
    STR_MotorExceptionMA m_MotorException;
    #endif

    STR_RampGen32        m_STR_CurRampGen;

    //
    STR_CurrentAmplitude m_struCurrentAmplitude;    // 电流幅值
    STR_VectorUVW        m_struImaxCurrent;          // 电流最大值
    STR_FaultVariable    m_stru_FaultValue;

    STR_FaultStatus      m_stru_Faults;

    STR_FaultTime        m_struFaultVoltTime;
    STR_FaultTime        m_struFaultEmptyTime;
    STR_FaultTime        m_struFaultStallTime;
    STR_FaultTime        m_struFaultCurrentTime;
    STR_FaultTime        m_struFaultTempTime;
    STR_FaultTime        m_struFaultPhaseTime;
    STR_FaultTime        m_struFaultStartTime;
} STR_MASubBlock;
```

MA 控制对象数据结构

```
typedef struct
{
    /*-----
    // 绑定的控制对象
    u8          mg_nMACtrlProcID;          /* 控制对象 ID */
    PSTR_ControlObject mg_pCtrlObj;        /* 控制对象 */

    /*-----
    // FAULT 检测交换信息
    STR_FaultStatus    mg_UFault;
    ENUM_MotorBrakeMode mg_eFaultStopMode;
    ENU_FaultPrio      mg_eFaultPrio;

    /*-----
    // 内部交换信息存储及刷新缓冲区, 节省RAM时可以直接采用mg_pCtrlObj绑定的数据交
    STR_PubDatMDMA      mMdToMa;
    STR_PubDatMAMD      mMaToMd;

    STR_PubDatUAMA      mUaToMa;
    STR_PubDatMAUA      mMaToUa;

    /*-----
    // 内部控制变量 (私有)
    volatile ul6         m_nSys_TimerMA;    /* MA周期计数Cnt */

    STR_MACtrlBit        m_SMACtrlBit;      /* 控制命令及当前状态寄存器,
    .....
```

电机控制新平台软件简介

```
ENUM_MotorBrakeMode    m_eStopMode;          // 电机停止模式
u8                     m_nLoopMode;          /* 环路状态, 速度、功率、 */

ENU_DRIVER_STATUS      m_eSysState;          /* 当前电机运行状态机 */
ENU_DRIVER_STATUS      m_eNextSysState;      /* 下次电机运行状态机 */

u16                    m_nSetTimeLeftCnt;     /* 状态机等待时间计数器 */
u16                    m_nSetTimeLeftPrd;     /* 状态机等待时间周期 */

s16                    m_nInvTemp;
s16                    m_nBreakDuty;

s32                    m_wPower0001W;
s64                    m_wPower0001Wfil;
s16                    m_nPower0001WFactor;

// motor info 变量及配置参数
ENU_MOTOR_INFO_STATE   m_eMotorInfoState;    /* MOTOR INFO 状态 */
ENU_TRACKING_RESULT    m_eTrackingResult;     /* tracking结果 */

/*-----
// 各功能块数据空间 (私有)
PSTR_CommData          m_pAppCommData;       /* 运行流程控制结构体 */
PSTR_MechQuantity      m_pMotorSpeed;        /* 转速矢量结构体, core */

pSTR_OnTheFlyDetectMA   m_pOnTheFlyDetect;    /* 顺逆风启动结构体 */

#ifdef (U_UNDVOLT_PROT_ENABLE_M0 == 1)
pSTR_MotorExceptionMA  m_pMotorException;     /* 顺逆风启动结构体 */
#endif

PSTR_Time_t            m_pTime;              /* 定位时间等 时间计时 */

PSTR_RampGen32          m_pSTR_CurRampGen;

#ifdef ((CLOSE_LOOP_M0 == POWER_LOOP) || ((POWER_LIMIT_STATUS_M0 == TRUE) &&
pSTR_PowerDef          m_pPower;
#endif

/*-----
// Fault 数据空间 (私有)
STR_FaultStatus        *m_pFaults;

STR_FaultVariable      *m_pFaultValue;
s16                    t_nFlagCheck;
STR_CurrentAmplitude   *m_pstruCurrentAmplitude;
STR_VectorUVW          *m_pstruImaxCurrent;

STR_FaultTime          *m_pstruFaultVoltTime;
STR_FaultTime          *m_pstruFaultEmptyTime;
STR_FaultTime          *m_pstruFaultStallTime;
STR_FaultTime          *m_pstruFaultCurrentTime;
STR_FaultTime          *m_pstruFaultTempTime;
STR_FaultTime          *m_pstruFaultPhaseTime;
STR_FaultTime          *m_pstruFaultStartTime;

/*-----
s16                    m_nQCur_Reference;    /* Q轴电流环参考给定 */
s16                    m_nDCur_Reference;     /* D轴电流环参考给定 */
s16                    m_nQCurrentSet;       /* Q轴电流设定值 */
s16                    m_nDCurrentSet;       /* D轴电流设定值 */
STR_MACtrProc, *PSTR_MACtrProc;
```

对于未使用的数据可以删除

7. MD 层控制对象

```
STR_MotorFoc          gS_FocObjM[MAX_PWM_MODULE_USED];
STR_MotorFocElement    gS_FocElementObjM[MAX_PWM_MODULE_USED];
#if (MAX_PWM_MODULE_USED == 1)
const STR_MotorFocHandle gS_MotorFocHandleM[MAX_PWM_MODULE_USED] = {
    Motor_FocMethod_M0
};

#elif (MAX_PWM_MODULE_USED == 2)
const STR_MotorFocHandle gS_MotorFocHandleM[MAX_PWM_MODULE_USED] = {
    Motor_FocMethod_M0,
    Motor_FocMethod_M1
};

#endif
```

MD 层控制库未根据 **MAX_PWM_MODULE_USED** 配置进行宏定义

库封装的变量及实例在库文件中包含，链接时自动分配内存。

MD 层处理状态：

```
typedef enum
{
    E_MOTOR_RESET,
    E_MOTOR_TRACKING,
    E_MOTOR_ALIGN,
    E_MOTOR_ASYN,
    E_MOTOR_PRE_SYN,
    E_MOTOR_SYN,
    E_MOTOR_BRAKE,
    E_MOTOR_IPD,
    E_MOTOR_OFFLINE_ID
} ENUM_MotorStatus; //MOTOR_STATUS;
```

其中：

1) E_MOTOR_RESET 状态

此状态下，MOE 是否使能根据 MA 与 MD 层交互数据的 ePwmCmd 命令决定，输出的脉宽也有 MA 传递到 MD 层脉宽数值决定

此外，其他控制命令由 nuFineCmd 中定义，MA 传输控制命令到 MD 层，MD 层实现控制命令操作，包括：电流环初始化、观测器参数计算、观测器初始化、顺逆风初始化、IPD 初始化、预充电处理、过流报警检测及清除操作。

2) E_MOTOR_TRACKING 状态

顺逆风检测，检测结果的频率值和电压值通过复用的 MD 与 MA 数据交换区返回给 MA 层，由 MA 进行后续处理

3) E_MOTOR_ASYN、E_MOTOR_ALIGN、E_MOTOR_PRE_SYN、E_MOTOR_SYN 状态

电流环及观测器运行

4) E_MOTOR_BRAKE 状态

短接制动状态

电机控制新平台软件简介

5) E_MOTOR_IPD 状态
IPD 检测代码

为缩短中断子程序执行时间，使能 QUICK_CURRENTKLOOP_DUNCTION 宏定义时，函数 MDtaskHandleIn1msLoop () 可运行在 1ms 任务中。

gS_MotorFocHandleM[] 主要定义 FOC 的实现函数函数指针，封装后通过函数调用实现 FOC 功能，目前主要用于 SVPWM 函数调用，其余函数在代码中通过对象 ID 直接调用。

FOC 控制对象数据结构详见 STR_MotorFoc 和 STR_MotorFocElement。

8. HD 层配置

当前代码基于 EVM 板，底层驱动代码的封装及配置，在 HAL_DEF.c 文件中。

在 MD 与 HD 的交互数据定义头文件 PubData_HD_MD.h 有详细的定义：

```
//定义MD ---->HD 驱动调用函数接口
typedef struct{
    void (*AcquireCurrent) (STR_PubDatHDMD* tS_pCurAdcRes);
    void (*AcquireVdc) (STR_PubDatHDMD* tS_pCurAdcRes);
    void (*AcquireNTC) (STR_PubDatHDMD* tS_pAdcRes);
    void (*AcquireBemf) (STR_PubDatHDMD* tS_pAdcRes);
    void (*AD_ChannelUpdate) (STR_PubDatMDHD* tS_pMdToHd);
    void (*MCPWMx_RegUpdate) (STR_PubDatMDHD* tS_pMdToHd);
    void (*PwmOutEnable) (FuncState t_state);
    void (*PwmCharge) (u8 t_bphaseABC);
    void (*InitPwmChargeEnd) (void);
    void (*GetEPWMBreakIn) (STR_PubDatHDMD* tS_pHd2Md);
    void (*ClrEPWMBreakIn) (void);
}STR_HALDrvHandle,*PSTR_HALDrvHandle;

#if (MAX_PWM_MODULE_USED == 1)
const STR_HALDrvHandle gS_MotorHalDrvHandleM[MAX_PWM_MODULE_USED] = {Motor_HALDrv_EPWM0
};
#elif (MAX_PWM_MODULE_USED == 2)
const STR_HALDrvHandle gS_MotorHalDrvHandleM[MAX_PWM_MODULE_USED] = {Motor_HALDrv_EPWM0,
Motor_HALDrv_EPWM1
};
#endif

#define Motor_HALDrv_EPWM0 \
{ \
    /* DRV handle */ \
    AcquireCurSamplingResultsM0, /* 读取电流AD采样 */ \
    AcquireVdcSamplingResultsM0, /* 读取母线AD采样 */ \
    AcquireNTCSamplingResultsM0, /* 读取NTC AD采样 */ \
    AcquireBmefSamplingResultsM0, /* 读取BMEF AD采样 */ \
    ChangeADChanelCFG0, /* 切换AD采样通道 */ \
    MCPWM0_RegUpdate, /* PWM加载 */ \
    EPWM0_OutPut, /* PWM输出使能 */ \
    EPWM0_Charge, /* PWM预充电 */ \
    InitEPWM0ChargeEnd, /* PWM预充电初始化 */ \
    GetEPWM0_breakInStatus, /* 读取 BreakIn 状态 */ \
    ClrEPWM0_breakInStatus /* 清除 BreakIn 状态 */ \
}
```

具体的驱动函数和硬件配置，在 HD 层相关文件中修改实现，MD 层通过封装的函数指针调用，实现 MD 与 HD 层的隔离。

9. 数据格式

当前的 08 MCU 平台基于 cortex M0+内核，为减少代码空间、提高执行速度，内部的数据计算采用定点 16 位数据格式。

平台代码中部分变量虽然定义为 32 位数据，但实际运算时大部分采用 16 位数据格式，实际应用时可对应采用的标么格式进行数据格式更改，减少占用的 RAM 空间。

03x/07x/45x 采用定点数处理时，采用相同的数据格式，代码风格、实现方式等保持与 03 MCU 平台代码的统一。

9.1 标么定义

当前的 MCU 内部集成 12 位 AD 运算放大器，转化结果推荐统一使用左对齐方式。AD 量程设置为 2.4V 时如下图所示，详见 LKS32MC03x_User_Manual。

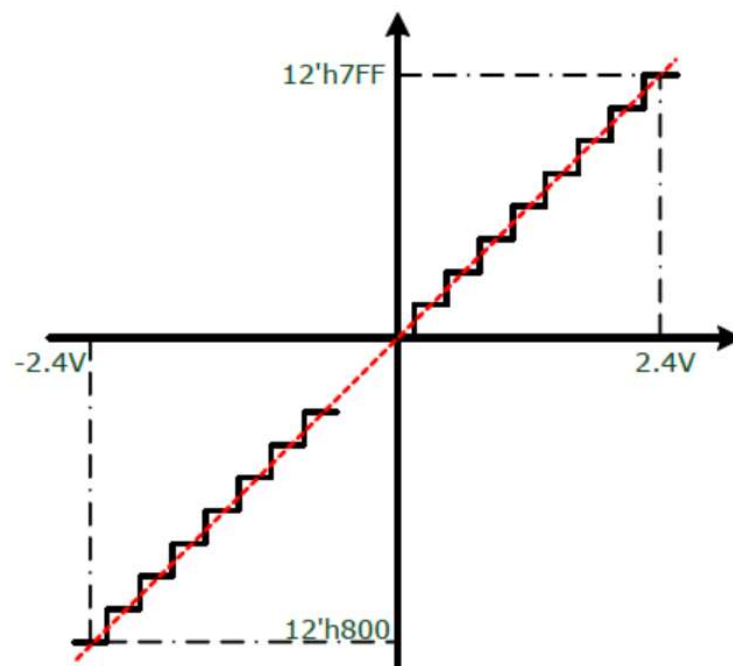
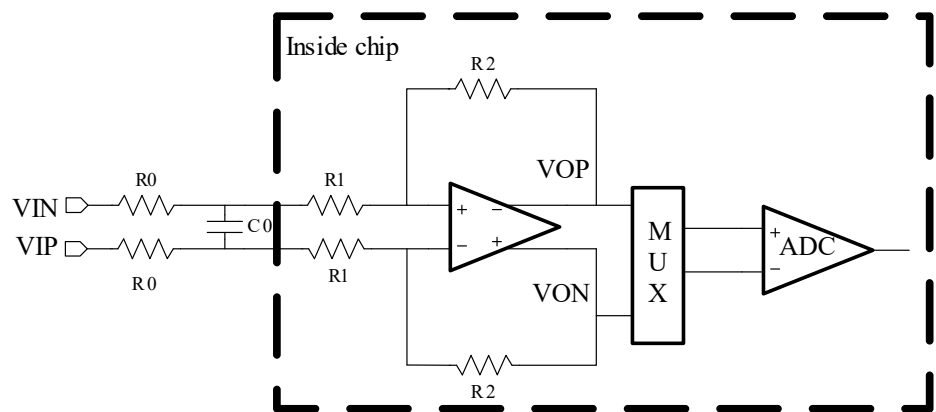


图 12-2 ADC 2.4V 量程设置下转换数制量程

AD 转换结果的数据范围为 $-2^{11} \sim (2^{11}-1)$ ，左移 4 位后范围为 $-32768 \sim 32752$ 。

为减少计算量，程序中的数据采用 Q15 格式。

9.1.1 电流标么



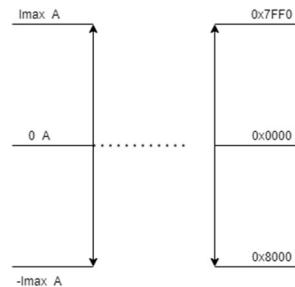
两个 R0 是片外需放置的电阻，阻值必须相等，最终的放大倍数为

$$\text{Gain} = R2 / (R1 + R0)$$

ADC 的输入信号范围最大为 $\pm 3.6V$ ，反馈电阻 R2:R1 的阻值可通过寄存器 RES_OPAX[1:0] 设置，以实现不同的放大倍数。

因此采样电路的最大采样电流 $I_{\text{max}} = 3.6 / (\text{Gain} * R_{\text{shunt}})$
其中 R_{shunt} 为采样电阻阻值。

由于硬件电路的偏置等原因可能会导致差分采样时，整个采样电路能采集到的正向最大采样电流 $I_{\text{max_pos}}$ 和负向最大采样电流 $I_{\text{max_neg}}$ 会有微小的差异，并且在采样结果左移四位后也有略微的数据不同，在允许的误差范围之内，为简便处理，统一表示为 I_{max} 对应 Q15 数据的 0x7FF0，即采用如下的标么格式



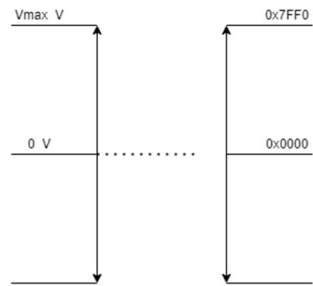
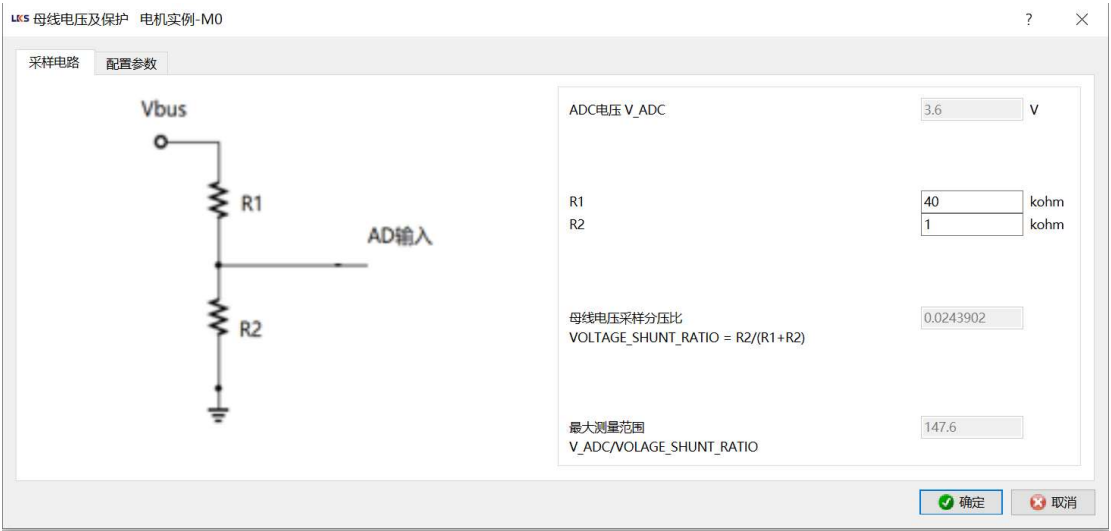
通过采用最大值进行标么后，AD 的采样结果即为 Q15 的标么数据，程序直接读取 AD 的采样结果进行数据处理。

45x 系列 AD 内部参考电压为 3.3v，处理方法预与其他 MCU 相同。

9.1.2 电压标么

根据母线电压采样方式，计算最大的电压采样范围

$$V_{max} = V_{adc} \cdot (R1 + R2) / R2$$



当反电势采样与母线采样电路不同时，采用各自的标么格式，数据需要转换时进行 AD 采样数据的处理，变换为统一数据格式，程序中需要添加相应的处理。

9.1.3 速度标么

基于当前电机运行最高频率，选定合适的最高频率 fmax 作为基准值，即 IQ15(1),然后进行相应的数据标么处理，最终所有表示的频率数据均为 16 位数据。

9.1.4 角度标么

基于电角度，360°为 65536，进行相应的数据标么处理，角度作为有符号数时表示

电机控制新平台软件简介

为：

$0 \sim 180^\circ$	:	$0 \sim 32767$
$-0 \sim -180^\circ$	:	$-0 \sim -32768$

角度作为无符号数时， $0 \sim 360^\circ$ 对应 $0 \sim 65536$

9.2 标么函数

当前的代码分为用户层、电机应用层、电机控制层。
针对当前程序中采用的数据，用户层数据为 User Data，电机应用层数据为 App Data，电机控制层为 Core Data。

其相应的数据关系如下图表格所示。

物理量	User Data	App Data		Core Data	
	实际单位(float)	表示数值	单位刻度	表示数值	单位刻度
电压	1.0V	100	0.01V	$32767 * 100 / (100 * U_{max})$	32767/ U_{max}
	U_{max}	$100 * U_{max}$		32767	
电流	1.0A	1000	0.001A	$32767 * 1000 / (1000 * I_{max})$	32767/ I_{max}
	I_{max}	$1000 * I_{max}$		32767	
频率	1.0Hz	100	0.01Hz	$32767 * 100 / (100 * f_{max})$	32767/ f_{max}
	f_{max}	$100 * f_{max}$		32767	
功率	1.0W	100	0.01W	$32767 * 100 / (100 * P_{max})$	32767/ P_{max}
	$P_{max} = U_{max} * I_{max}$	$100 * P_{max}$		32767	

用户层数据即 User Data 采用 float 数据时，代码编译后会引入浮点库，占用较大的 FLASH 空间。
因此，默认用户层采用 App Data 的数据格式进行处理。所有的配置参数对外显示表示为 float 数据，实际程序中应用时通过宏定义或者通过 LKSMotor 转换为定点数进行处理。

App Data 到 Core Data 的转换函数

```
extern s32 App2CoreCurTrans(STR_TransCoefElement* pApp2CoreCur,s32 val);
extern s32 App2CoreFreqTrans(STR_TransCoefElement* pApp2CoreFreq,s32 val);
extern s32 App2CoreVoltTrans(PSTR_TransCoefElement pApp2CoreVolt,s32 val);
```

Core Data 到 App Data 的转化函数

```
extern s32 Core2AppCurTrans(STR_TransCoefElement* pCore2AppCur,s32 val);
extern s32 Core2AppFreqTrans(STR_TransCoefElement* pCore2AppFreq,s32 val);
extern s32 Core2AppVolTrans(STR_TransCoefElement* pCore2AppVol,s32 val);
extern s32 Core2AppPowerTrans(PSTR_TransCoefElement pCore2AppPower,s16 val);
```

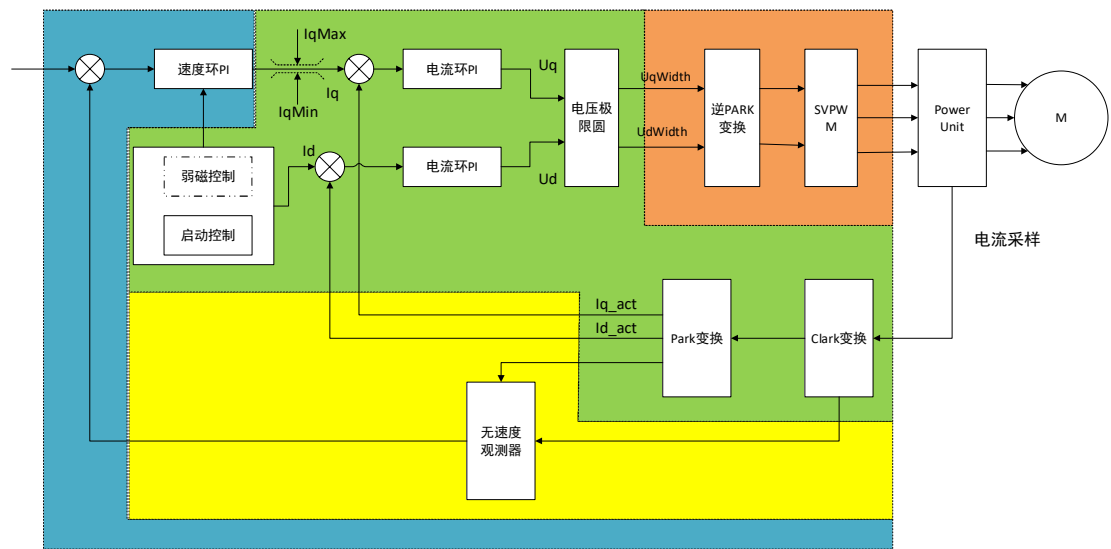
程序中包含预留的浮点数据转换函数接口，用户在接受占用较大 FLASH 空间的前提下可以调用浮点标么转换函数进行数据处理。

预留的 User Data 到 App Data 的浮点转换函数：

```
extern s32 User2AppCurTrans(STR_TransCoefElement* pUser2AppCur,float val);
extern s32 User2AppFreqTrans(STR_TransCoefElement* pUser2App,float val);
extern s32 User2AppVolTrans(PSTR_TransCoefElement pUser2AppVolt,float val);
```

9.3 数据表示

在如下的简要控制框图中，用不同的色块对整个控制框图进行了大概的划分。



区域中，

Core Data 数据表示为下面的映射关系, App Data 数据需按照约定的单位刻度进行标定, 如 1A 表示为 1000 等。

输入量

频率或转速

$$\begin{bmatrix} f_{\max} \\ 0 \\ -f_{\max} \end{bmatrix} \longleftrightarrow \begin{bmatrix} 32767 \\ 0 \\ -32767 \end{bmatrix}$$

输出控制量

电流

$$\begin{bmatrix} I_{\max} \\ -I_{\max} \end{bmatrix} \longleftrightarrow \begin{bmatrix} 32767 \\ 0 \\ -32767 \end{bmatrix}$$

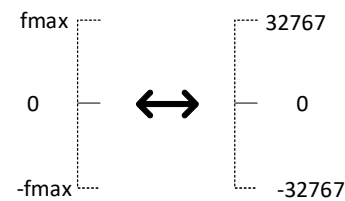
、 区域中，

Core Data 数据表示为下面的映射关系， App 数据按照单位刻度进行标定。

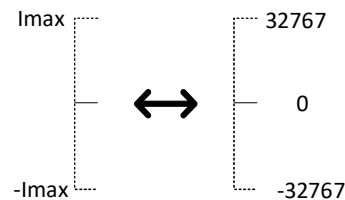
电机控制新平台软件简介

输入量

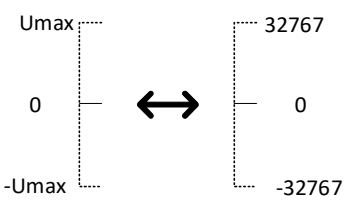
频率或转速



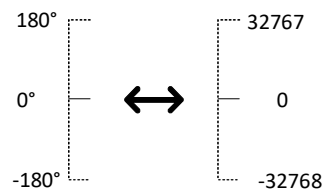
电流



母线电压

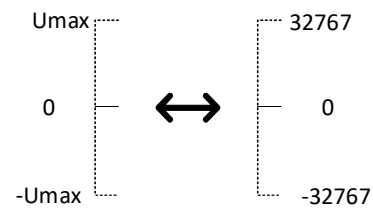


角度



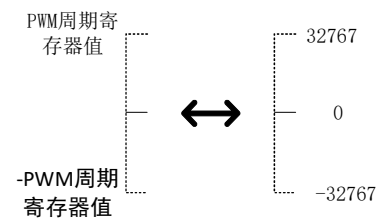
输出量:

dq 电压及占空比



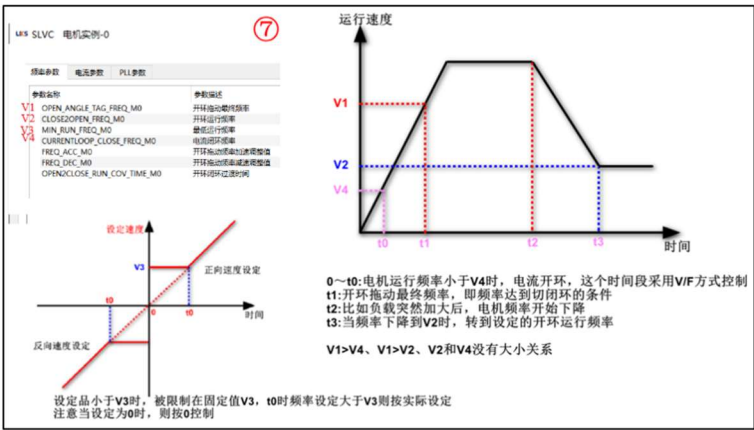
区域中,

Core Data 数据表示为下面的映射关系



10. 功能简介

10.1 VF 启动



通过配置电流闭环参数数值，在给定频率小于该设定值时实现 VF 启动（电机参数中的电阻、电感、反电势系数须与实际电机对应）。

给定频率大于电流闭环参数设定值时，自动切入电流闭环控制。

所有参数可配置，可以通过修改代码的参数设置或者通过 LksMotor 配置参数实现。

10.2 自动过调制

自动过调制（可设定最大过调制电压），最大到达母线电压的 2/3 输出。

电压输出最大值大小取决于电流采样方式，双电阻时为保证可靠的采样电流，在部分扇区无法到最大输出幅值。

过调制时自动实现 7 段式/5 段式调制方式切换，电压满足需要时默认采用 7 段式调制。

当前默认最大过调制的调制系数在 MC_GlobalDef_Mx.h 文件中可以进行配置，默认最大过调制。

```
#define OVER_MODU_FACTOR_M0 ((s16)21845) /*单位: Q15, 过调制系数*/  
21845 = INT(32767*2/3)
```

实际项目中可以根据需要进行修改。

增加死区补偿功能：

$$T_{comp} = T_{dead} + T_{on} + T_{on_popdelay} - (T_{off} + T_{off_popdelay})$$

通过功率器件导通关断时间及死区配置时间、以及传输延时时间，计算出死区补偿时间 T_{comp}

$$\text{则补偿系数为 } factor = \frac{T_{comp}}{T_{pwm}} * 65536$$

系数最大设置值为 32767

10.3 SVPWM

对双电阻和三电阻，基于执行时间和代码空间提供多种 SVPWM 调制方式，可根据需要选择合适的调制方式。

对单电阻，可采用噪声优化模式。
参见相关库函数。

10.4 自动弱磁

自动弱磁功能可以通过编译宏决定是否编入代码。

通过编译宏编译后，可以设置最大弱磁电流数值决定弱磁是否起作用（设置最大弱磁电流为 0 时，弱磁功能实际不起作用）。

对于弱磁功能要求严格的，例如在当前电压下达到最大弱磁扭矩输出，后续实现动态最大弱磁电流调整功能。

弱磁参数的调整及设置需要根据实际应用进行调试。

弱磁配置可以查看相关库函数。

10.5 参数自动计算

电流环参数、速度环参数可自动计算，用户可以配置是否使用计算值。

10.6 无速度观测方式

基于执行时间的优化，可选择合适的无速度观测模型，后续提供其他观测器及观测算法的升级

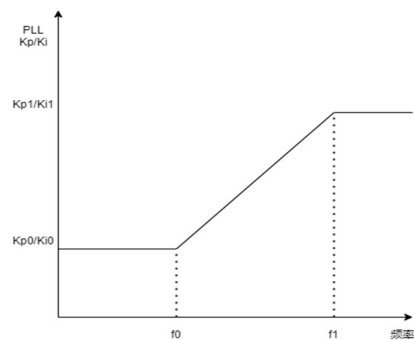
10.7 电流环解耦补偿

增加电流环解耦补偿功能，适当修改代码可实现类似与 D 轴电流闭环、Q 轴开环控制的方式等功能。

10.8 快速闭环启动

降低闭环速度设定值、设置较快的开环加速度，设定 I_d 电流启动值，可以实现不对相、不打初始位置情况下快速的闭环启动，在不使能顺逆风功能下可以直接顺风启动、逆风启动。

10.9 锁相环参数调整



设置锁相环参数的调整区间及调整数值，可以实现锁相环参数的动态调整。

10.10 母线电压保护功能

电机运行过程中，可以配置使能母线电压保护功能，防止在运行过程中母线电压丢失导致的不安全状况发生，详见库函数。

详细内容参见后续的库函数简介。

11.库函数

本章节简述封装库提供的函数接口及功能, 开发人员可以通过相应的函数实现所需要的功能。

函数中 t_bObjId 为控制对象 ID, 通过此 ID 可查找到所需要的内部信息。

库支持的函数及功能具体详见 mc_declaration.h 文件。

11.0 状态检查函数

extern bool getRunStatus(u8 t_bObjId)

查看是否进入速度闭环状态, True ----- 速度闭环

extern bool getCurLoopCloseStatusFromOB(u8 t_bObjId)

查看是否进入电流闭环状态 True ----- 电流闭环 False ----- 电流开环

11.1 VF 函数

VF 启动用于硬件调试或者 VF 启动（单电阻、双电阻以及三电阻）。

在不修改代码的前提下, 用户可以通过参数配置实现电压随频率近似线性变化的功能, 从而实现 VF 启动, 同时可采集电流采样信号检验硬件功能是否正常。

配置参数, 下图所示的 LKSMotor 配置参数（代码中配置参数完全相同）:

VF 控制需要的电机参数, 为达到较好的 VF 效果, 电机参数要尽量准确。

参数名称	参数描述	数值	量纲
U_MOTOR_PP_M0	极对数	5	对
U_MAX_FREQ_M0	额定频率	1333.00	1Hz
U_MOTOR_RS_M0	电机相电阻 (25°C)	0.4442	1 ohm
U_MOTOR_LD_M0	电机D轴电感	234	uH
U_MOTOR_LQ_M0	电机Q轴电感	300	uH
U_MOTOR_FLUX_CONST_M0	电机磁链常数	0.01747	1WB

VF 控制配置参数,

参数名称	参数描述	数值	量纲
OPEN_ANGLE_TAG_FREQ_M0	开环拖动最终频率	8.00	1Hz
CLOSE2OPEN_FREQ_M0	开环运行频率	3.00	1Hz
MIN_RUN_FREQ_M0	最低运行频率	30.00	1Hz
CURRENTLOOP_CLOSE_FREQ_M0	电流闭环频率	0.00	1Hz
FREQ_ACC_M0	开环拖动频率加速调整值	10.00	1Hz/s
FREQ_DEC_M0	开环拖动频率减速调整值	10.00	1Hz/s
OPEN2CLOSE_RUN_COV_TIME_M0	开环闭环过渡时间	30	1ms

选择合适的电流闭环频率 f2 以及开环拖动最终频率 f1, 并设置 f1>f2 时。

a) 当给定的频率值 f < f2 时, 实现 VF 的拖动功能, 电流闭环不工作。

- b) 当给定频率值 $f \geq f_2$ 时，电流闭环开始工作，VF 功能是实现解耦补偿。当不需要进行解耦补偿时，可以通过代码或者宏编译去除 VF 功能。

VF 函数：

extern void CalcVectorVFPara(u8 t_bObjId)

计算 VF 控制参数，在初始化中完成。

extern void setDeadCompFactor(u8 t_bObjId, s16 factor)

设置 VF 控制的死区补偿系数，如果已经进行死区补偿，此函数可以忽略。

factor 为补偿系数，IQ15 格式，开关频率 16K 死区 1us 时默认为 524。

选用不同的功率器件时需要根据功率器件选择合适的补偿数值。

extern void vectorVFcontrol(u8 t_bObjId, s16 *pUd, s16 *pUq)

VF 控制函数或者 DQ 补偿函数，pUd 为 d 轴电压变量指针，pUq 为 q 轴电压变量指针。

该函数通过指针回传的方式实现 dq 轴电压的控制或补偿。

11.2 电流设置及弱磁控制函数

设置电机的启动电流，弱磁控制等的控制参数

**extern void InitDQCurRefGen(u8 t_bObjId, s16 dCurSet,
s16 qCurSet, s16 TransTime);**

不带弱磁控制时 DQ 轴电流指令初始化函数

其中：dCurSet	----- d 轴启动电流	(input)
qCurSet	----- q 轴启动电流	(input)
TransTime	----- 开环闭环切换过渡时间	(input)

**extern void InitDQCurRefGenWithWeakenFlux(u8 t_bObjId, s16 dCurSet,
s16 qCurSet, s16 TransTime)**

带弱磁控制时 DQ 轴电流指令初始化函数

其中：dCurSet	----- d 轴启动电流	(input)
qCurSet	----- q 轴启动电流	(input)
TransTime	----- 开环闭环切换过渡时间	(input)

extern void SetDCurRefInitValue(u8 t_bObjId, s16 dCurref)

用于设定 D 轴电流指令初始值

其中：dCurref	----- 电流设定值	(input)
------------	-------------	---------

extern s16 getDCurRefGen(u8 t_bObjId)

电机控制新平台软件简介

不带弱磁控制时获取 d 轴电流指令值

extern s16 getDCurRefGenWithEnWeakenFlux(u8 t_bObjId)

带弱磁控制时获取 d 轴电流指令值，该函数包括自动产生的弱磁电流值

extern void QCurRefGenValue(u8 t_bObjId, s64 *pValue,u8 QFormat,s32 spdSet)

获取启动时的 q 轴电流

其中： QFormat ----- 回传数据格式 (input)
*pValue ----- 根据数据格式回传的电流数值 (output)
spdSet ----- 预留

extern void setFieldWeakenPara(u8 t_bObjId, u16 Kp, u16 Ki)

设置弱磁控制 PI 参数，弱磁控制时根据需要调整弱磁控制比例积分系数

其中： Kp ----- 比例系数，默认为 0 (input)
Ki ----- 积分系数，默认为 3276 (input)

extern void setOverModuFac(u8 t_bObjId, s16 factor)

设置过调制系数，根据需要可重新配置或者设置

其中： factor ----- 21845，最大过调制
18919，内切圆 0.866 调制

extern void setFieldWeakenVoltFac(u8 t_bObjId, s16 factor)

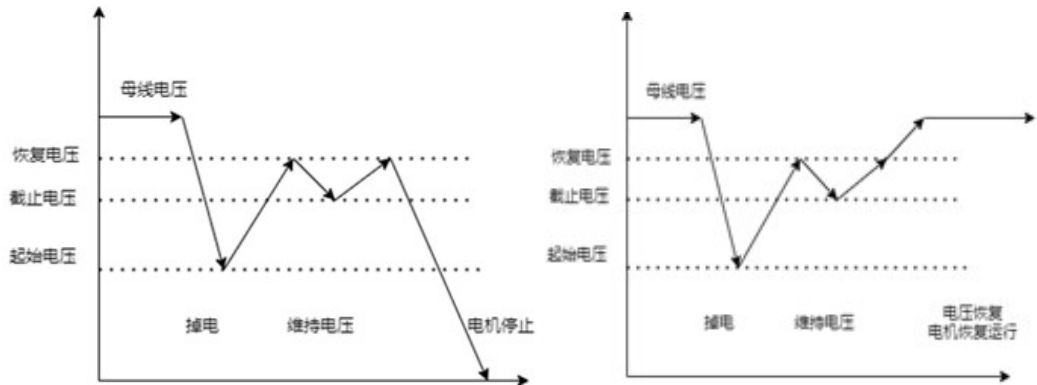
设置弱磁控制电压利用系数，根据需要可重新配置或者设置

其中： factor ----- 4096，100%

11.3 低压保护异常处理函数

电机运行过程中出现异常，如掉电或者母线电压变低等异常时，库中已经包含了进行电机控制和母线电压维持的处理函数，用户可以编入相应代码实现所需的功能。

电压保护如下，左图为运行中电压丢失，右图为运行中电压丢失然后恢复



处理函数:

其中: RunStop ----- 当前 运行/停止 命令 1 ----- Run (input)
0 ----- Stop
Stop 时如果关闭 MOE, 则该函数无效
PReturnCurSet ----- 返回 q 轴电流指令值。 (output)

电机控制新平台软件简介

观测器进行无速度观测运行函数，低速好于 **PmsmFluxObserveSimplyxxx**，

执行时间长于 **PmsmFluxObserveSimplyxxx**

其中 tS_pCurDq ----- dq 轴 实际电流 (input)

extern void PmsmFluxObserve_Full(u8 t_bObjId,PSTR_VectorDQ tS_pCurDq)

低速性能优于 **PmsmFluxObserve**，执行时间略长，不能大范围弱磁

extern void PmsmFluxObserveIDWeaken(u8 t_bObjId,PSTR_VectorDQ tS_pCurDq)

低速性能优于 **PmsmFluxObserve**，执行时间最长，适用于大范围弱磁

extern u16 getElectAngle(u8 t_bObjId)

获取当前的电角度

extern u16 getElectAngleEst(u8 t_bObjId)

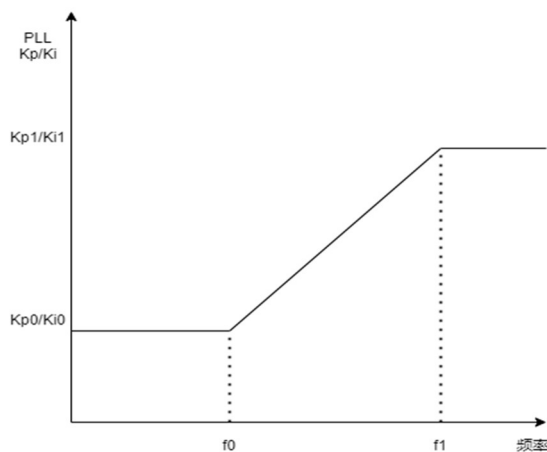
获取当前的观测器估计角度

extern void setObserPLLKpKi(u8 t_bObjId,u16 Kp, u16 Ki);

设置观测器 PLL 参数

代码中已经添加 PLL 参数调整功能函数，用户可以根据需要进行调整。

void updateObserPLLVar1msLoop(PSTR_MotorFoc tS_pMotorFoc)



extern void PmsmFluxObserve1ms(u8 t_bObjId)

观测器 1ms 任务处理函数

extern void setOBElectAngleEst(u8 t_bObjId, u32 theta)

设置观测器角度

其中 θ ----- 电角度 (input)

0xFFFFFFFF 为 360°

extern void CalcVolCircle(u8 t_bObjId,

PSTR_VectorDQ tS_pVolDq,

PSTR_VectorAB tS_pVolAB);

电压极限园处理函数，函数中已经内嵌过调制处理

其中: tS_pVolDq ----- dq 电压 (input)

tS_pVolAB ----- alph/beta 占空比 (output)

11.5 顺逆风检测函数

LKSMotor 参数配置:

参数名称	参数描述	数值	量纲
SPEED_TRACK_DELAYTIME_M0	顺逆风检测时间	500	ms
SPEED_TRACK_ON_FREQ_THH_M0	顺风切闭环频率	25.00	1Hz
EBRK_ON_FREQ_THH_M0	逆风刹车频率	25.00	1Hz
MOTOR_STOP_CUR_THD_M0	电机停转电流检测阈值	0.20	A
MOTOR_STOP_CUR_DIF_THD_M0	电机停转电流差值检测阈值	0.10	A
STOP_TIME_M0	电机停止检测滤波时间	200	ms
STOP_DELAY_TIME_M0	电机停止后延迟时间	200	ms
BEMF_ANG_COMP_M0	反电势角度补偿	0	度
BEMF_TRACK_MIN_VOLT_M0	反电势检测最小电压	200	0.01V

处理函数

```
extern void OnTheFlyDetectInit(u8 t_bObjId);
    顺逆风检测变量初始化函数

extern void OnTheFlyDetectPro(u8 t_bObjId)
    顺逆风检测运行函数

extern void setTrakingModeEnable(u8 t_bObjId);
    使能顺逆风观测功能

extern void setTrakingModeDisable (u8 t_bObjId);
    不使能顺逆风观测功能
```

在运行观测 PmsmFluxObserve () 前需要判断是否使能顺逆风功能

11.7 IPD 函数

LKSMotor 参数配置:

参数名称	参数描述	数值	量纲
U_START_ANGLE_COMP_M0	初始位置检测补偿角度	0.00	度
IPD_PLUS_TIME_SETTING_M0	脉冲注入时间宽度设置	30	1us
IPD_WAIT_TIME_SETTING_M0	脉冲空闲等待时间宽度设置	500	1us

```
extern void IPDEst_SMotor (u8 t_bObjId)
    利用 Hall cnt 计时的 IPD 检测函数，可用于单电机 IPD，使用前需要使能 HALL 外设模块

extern void IPDEst_DMotor (u8 t_bObjId)
    利用 PWM 周期计时的 IPD 检测函数，可用于双电机 IPD

extern bool getIPDFinishFlag(u8 t_bObjId)
    IPD 结束标志

extern u16 getIPDAngleLib(u8 t_bObjId)
    IPD 检测结果，65536 ----- 360°
```

11.8 SVPWM 函数

单电阻采样函数：

extern void SVPWM_1Shunt(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

单电阻 SVPWM 函数

tS_pVolAB ----- alph/beta 占空比 (input)

extern void SVPWM_1Shunt_Mode0(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

单电阻噪声优化模式 0 SVPWM 函数

低速优化噪声，高速采用 SVPWM_1Shunt 相同处理方式

extern void SVPWM_1Shunt_Mode1(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

单电阻噪声优化模式 1 SVPWM 函数

高低速均采用特殊移相的优化噪声方式

双电阻采样函数：

extern void SVPWM_2Shunt_En1(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

双电阻 SVPWM 函数，增强型 1 （基于基本型，矢量缩放）

extern void SVPWM_2Shunt_En1_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB);

双电阻 SVPWM 函数，增强型 1 （基于基本型，矢量缩放）5 段式

extern void SVPWM_2Shunt_En2(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

双电阻 SVPWM 函数，增强型 2 （基于增强型，矢量缩放），部分扇区采样方式不同于

SVPWM_2Shunt_En1 函数

extern void SVPWM_2Shunt_En2_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB);

双电阻 SVPWM 函数，增强型 2 （基于增强型，矢量缩放）5 段式

三电阻采样函数：

extern void SVPWM_3Shunt(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

三电阻 SVPWM 函数，基本型

extern void SVPWM_3Shunt_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

三电阻 SVPWM 函数，基本型 5 段式

extern void SVPWM_3Shunt_En(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

三电阻 SVPWM 函数，增强型

extern void SVPWM_3Shunt_En_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB)

三电阻 SVPWM 函数，增强型 5 段式

11.9 PI 参数自计算函数

```
extern void SpeedPIAutoTunning(u8 t_bObjId,s16 KpFactor, s16 Ton,
                               s32 *pKp, s32 *pKi);
    其中, KpFactor      ----- Kp 增益系数, 1000 代表 1.0      (input)
           Ton          ----- 速度滤波时间 0.1ms          (input)
           s32 *pKp      ----- 回写 Kp 地址  Q9 格式          (output)
           s32 *pKi      ----- 回写 Ki 地址  Q14 格式        (output)
```

```
extern void SetCurPIPara(u8 t_bObjId,s32 Kpd, s32 Kid,s32 Kpq, s32 Kiq)
    设置电流环 PI 参数
```

11.10 反电势电路检测函数

```
extern void Bemf_CheckInit(u8 objId , u16 nCompAngle, u16 nBemfTrackVolt);

extern void Bemf_CheckInit(  u8 objId ,
                             u16 nCompAngle,
                             u16 nBemfTrackVolt,
                             u16 nBemfTrackVoltRecover)

    反电势检测初始化
    其中:   nCompAngle      ----- 补偿角度, 1 度          (input)
           nBemfTrackVolt   ----- 最小检测电压, 0.01V      (input)
           nBemfTrackVoltRecover ----- 最小检测恢复电压, 0.01V  (input)
                                           ----- V2.1.0a 版本后添加
```

```
extern void Bemf_CheckProcess(  u8 objId,
                                s16 phaseC,
                                s16 phaseB,
                                s16 phaseA)

    反电势检测函数
    其中: phaseA/B/C      ----- 反电势检测 AD 原始值      (input)
```

```
extern void Bemf_CheckProcessEn( u8 objId,
                                  s16 phaseC,
```

**s16 phaseB,
s16 phaseA)**

反电势检测增强函数

采用线电压检测的方式，增强对采样 offset 的冗余处理能力

V2.1.0a 版本后添加

其中：phaseA/B/C ----- 反电势检测 AD 原始值 (input)

extern void BemfSwitch (u8 objId, bool isWeakenUsed)

反电势检测后切换处理函数，isWeakenUsed 为 TRUE 时表明使用弱磁观测器

11.11 有感处理函数函数

采用有感处理时，可通过下面的函数设置角度、速度以及运行状态到库中，进行 FOC 运行处理。

extern void setEstSpdFromExternalModule(u8 t_bObjId, s32 estSpd);

设置观测速度函数

用于直接设置库计算的速度值，例如外部观测器或者传感器获得速度值后，可通过该函数直接设置库应用的速度设定值

其中：s32 estSpd ----- 速度标么值

extern void setOBElectAngleEst(u8 t_bObjId, u32 theta)

设置观测器角度

其中 theta ----- 电角度 (input)

0xFFFFFFFF 为 360°

extern void setVCState2RunStatus(u8 t_bObjId, bool iscloseLoop)

设置闭环状态到库模块

11.12 参数查看函数

u16 getElectAngle(u8 t_bObjId)

u16 getElectAngleEst(u8 t_bObjId)

s16 getElectSpdEst(u8 t_bObjId)

数据格式对应标么值，Iq15(1) 对应于最大频率

s32 getVolt2FreqCalcPara(u8 t_bObjId)

读取电压频率转换函数，电机当前电压对应电机频率近似计算

观测器变量查看：

s16 getThetaFilFromObjId(u8 t_bObjId)

观测器角度误差，标么值，Iq15(1)对应 180°

s16 getIdEstFromObjId(u8 t_bObjId)

观测器 d 轴电流估计值，标么值，Iq15(1)对应 Imax

s16 getIqEstFromObjId(u8 t_bObjId)

观测器 q 轴电流估计值，标么值，Iq15(1)对应 Imax

s16 getUdMrasInFromObjId(u8 t_bObjId)/ s16 getElectBmefD(u8 t_bObjId)

观测器 d 轴电压，标么值，Iq15(1)对应 Umax

s16 getUqMrasInFromObjId(u8 t_bObjId)/ s16 getElectBmefQ(u8 t_bObjId)

观测器 q 轴电压，标么值，Iq15(1)对应 Umax

s16 getUdMrasEstFromObjId (u8 t_bObjId)

观测器 d 轴电压估计值，标么值，Iq15(1)对应 Umax

s16 getUqMrasEstFromObjId (u8 t_bObjId)

观测器 q 轴电压估计值，标么值，Iq15(1)对应 Umax

s16 getMotorPowerFromObjId(u8 t_bObjId)

电机输入功率估算，标么值，Iq15(1)对应 $P_{max} = U_{max} * I_{max}$

基于电压基准值的电压变量：

s16 getMotorVoltFromObjId(u8 t_bObjId)

电压限幅前的幅值， $\text{SQRT}(u_d^2 + u_q^2)$

s16 getMotorDaxisVoltFormObjId(u8 t_bObjId)

当前 PWM 周期输出 d 轴电压

s16 getMotorQaxisVoltFormObjId(u8 t_bObjId)

当前 PWM 周期输出 q 轴电压

s16 getMotorActVoltFromObjId(u8 t_bObjId)

施加到电机端的实际电压， $\text{SQRT}(u_{d_act}^2 + u_{q_act}^2)$

基于电流基准值的电压变量：

s16 getMotorCurFormObjId(u8 t_bObjId)

相电流有效值

s16 getPhaseCur_U(u8 t_bObjId)

U 相电流值

s16 getPhaseCur_V(u8 t_bObjId)

V 相电流值

s16 getPhaseRawCur_U(u8 t_bObjId)

U 相电流 AD 值

s16 getPhaseRawCur_V(u8 t_bObjId)

V 相电流 AD 值

s16 getIalphCurrent(u8 t_bObjId)

电机控制新平台软件简介

I_{alph} 电流值

s16 getIbetaCurrent(u8 t_bObjId)

I_{beta} 电流值

s16 getIdCurrent(u8 t_bObjId)

I_d 电流值

s16 getIqCurrent(u8 t_bObjId)

I_q 电流值

反电势观测变量查看函数：

extern s32 getOnFlyBemfFreqResultFromObjId(u8 t_bObjId)

反电势顺逆风估算频率，标么值

extern s16 getBemfUVolt(u8 t_bObjId)

extern s16 getBemfVVolt(u8 t_bObjId)

extern s16 getBemfWVlt(u8 t_bObjId)

反电势电路 UVW 相电压，标压值

extern s16 getBemfAmpVolt(u8 t_bObjId)

反电势电路 UVW 相电压峰值，0.01V

IPD 检测查看函数：

extern bool getIPDFinishFlag(u8 t_bObjId)

IPD 结束标志

extern u16 getIPDAngleLib(u8 t_bObjId)

IPD 检测结果，65536 ----- 360°

检测结果未进行角度补偿

运行状态函数：

extern bool getCurLoopCloseStatusFromOB(u8 t_bObjId)

获取电流开闭环状态函数

extern bool getRunStatus(u8 t_bObjId)

获取 SLVC 运行状态，TRUE 无感闭环控制

电压极限圆查看函数：

extern s16 getVdcCirCleLim(u8 tobjId)

获取电压极限圆的电压值，标么值，IQ15(1)对应 U_{max}

extern s16 getMotorVoltFromObjId(u8 t_bObjId)

电流环输出的 dq 电压峰值，为经过电压极限圆处理，可用于弱磁控制

标么值，IQ15(1)对应 U_{max}

12. 报警处理内容

通用平台需要包含的报警：

- | | | |
|------------|---------------------------|---|
| 1) 过欠压报警, | 可配置电压及时间, 可使能/不使能 | 有 |
| 2) 缺相报警, | 可配置, 可使能/不使能 | 有 |
| 3) 硬件过流报警, | 内部比较器时可配置 DAC 数值 | 有 |
| 4) 软件过流报警, | 可配置电流值, 可使能/不使能 | 有 |
| 5) 堵转报警, | 可配置电流值, 可使能/不使能 | 有 |
| 6) | | |
| 7) 模块过温报警, | 可配置保护温度, 可使能/不使能, 只支持 NTC | 有 |
| 8) 启动失败报警 | 可配置 | |
| 9) 离水空转报警 | 可配置 | |

平台只提供最基本的硬件保护报警, 其他内容由用户自行实现

报警处理部分在 MA 层的 fault_detection.c 文件中, 报警处理信息接口已定义、不可修改, 报警实现方式用户可根据需要修改。

13. 调试步骤

调试步骤如下，详细信息参见相关调试手册。

Step1:

根据 MCU 类型，选择合适的 MCU 驱动库文件到 1_LKS_FwLib 文件夹

Step2:

根据 MCU 和原理图，修改 2_HardwareDriverLayer 文件夹下的硬件初始化函数和硬件抽象驱动函数

Step3:

根据 MCU 和原理图等，修改 Include 文件夹下的工程配置文件 project_config.h，硬件配置头文件 hardware_config.h，控制参数配置 MC_GlobalDef_M0.h 和 MC_Parameter_M0.h 等，实现需要功能的配置

Step4:

根据 MCU 和原理图等，检查报警检测代码及配置信息，完善必要的硬件保护代码，如过流、过压保护，防止硬件损坏。

\5_MotorAppLayer\usr\fault_detection.c

Step5:

设置控制参数配置 MC_GlobalDef_M0.h 和 MC_Parameter_M0.h 的相关参数，实现开环 VF 调试，电流闭环 IF 的调试，抓取采样电流等，检查硬件初始化和配置是否正确

Step6:

通过 UA 层的调试变量和命令设定值，实现 UA 层发送命令电机的转速闭环或电流环的控制。

抓取 UA 层变量的信息或者共用交换区的数据信息，进行调试分析

如有其他问题，依次抓取 MA 层变量、MD 层变量进行分析，并根据需要修改参数或者相应代码。

Step7:

在 UA 层添加用于实现用户功能的代码，如有必要修改 MA 层代码。